

GreenRoute

Plateforme de livraison éco-responsable du dernier kilomètre

Malik · Swann · Cléry

ESGI — Modélisation UML2 — 2025/2026

Soutenance orale — juillet 2026

github.com/cleeryy/GreenRoute-UML



Déroulé de la présentation

- 01 Besoins & cas d'utilisation**
Contexte, périmètre du système, diagramme de cas d'utilisation, descriptions textuelles
- 02 Architecture du modèle**
Diagramme de classes d'analyse, cycle de vie des commandes, diagrammes de séquence système
- 03 IHM & implémentation**
Maquettes web et mobile, critères ergonomiques de Bastien & Scapin, code Python du modèle
- 04 Bilan & questions**
Répartition du travail, pistes d'évolution, échanges avec le jury

Malik

Swann

Cléry

Tous

Contexte & problématique

Le problème : le dernier kilomètre à Toulouse

Les livraisons en centre-ville sont assurées par des **camionnettes thermiques** : saturation de l'espace urbain, pollution, coût environnemental élevé.

La solution GreenRoute

Orchestrer une flotte de **livreurs indépendants** en modes doux : vélos-cargos, triporteurs, camionnettes électriques. Projet lauréat d'un **appel d'offres de la métropole de Toulouse**.

Objectif 1 — Simplicité

Une plateforme web pour que les expéditeurs gèrent leurs livraisons simplement.

Objectif 2 — Intelligence

Répartir automatiquement la charge entre les livreurs disponibles (moteur d'attribution).

Objectif 3 — Impact mesuré

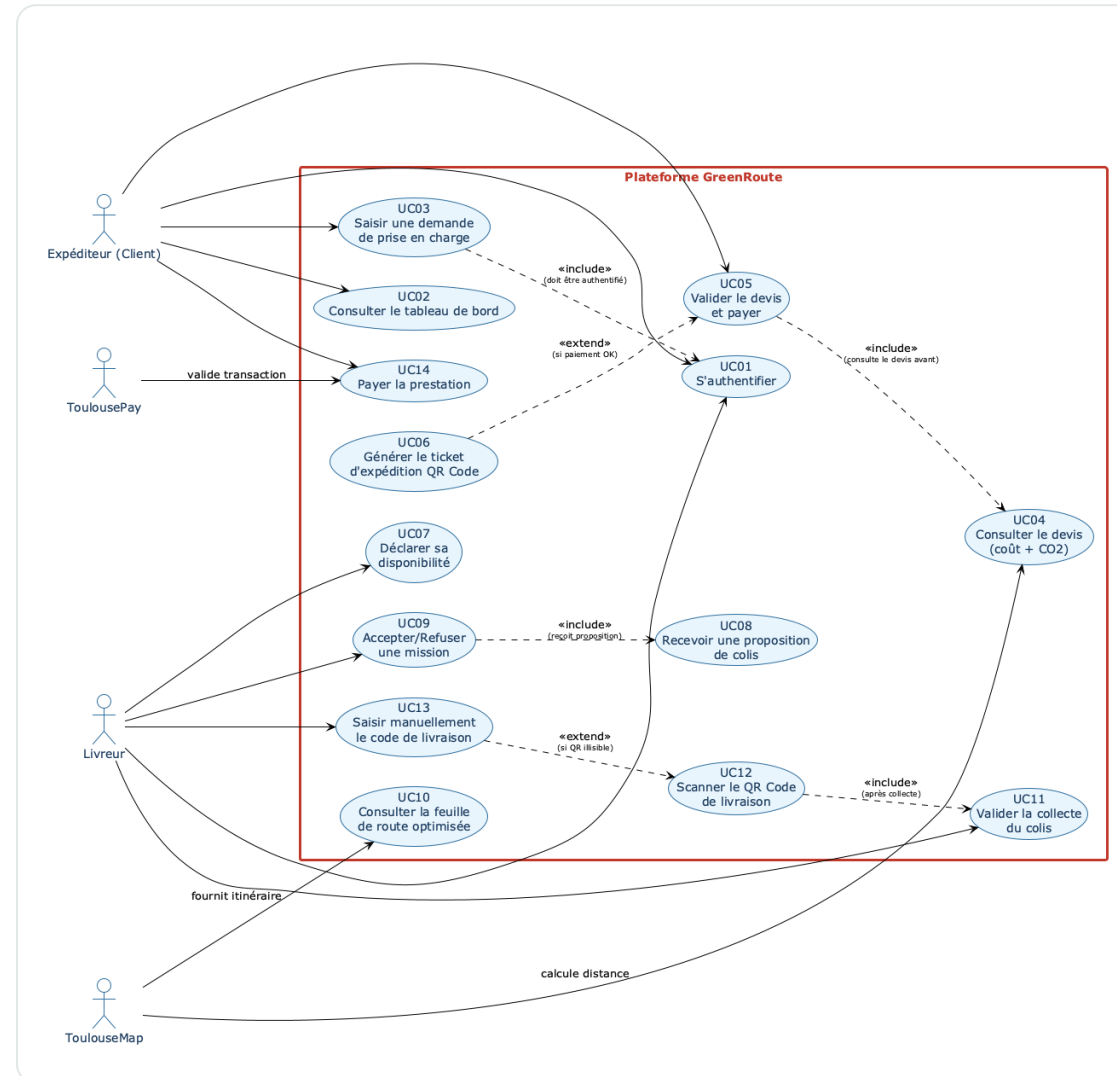
Prouver l'impact écologique de chaque trajet avec des métriques CO₂ (vs véhicule diesel).

Périmètre du système — diagramme de contexte



2 acteurs humains · 3 systèmes externes — le système GreenRoute est la frontière de responsabilité. Diagramme PlantUML complet dans le DAF (diagrams/contexte.puml).

Diagramme de cas d'utilisation



Côté Expéditeur (web)

S'authentifier · saisir une demande de prise en charge · consulter le devis · valider et payer · générer le ticket d'expédition QR Code.

Côté Livreur (mobile)

Déclarer sa disponibilité · recevoir une proposition · accepter/refuser · valider la collecte · scanner le QR Code de livraison (ou saisie manuelle).

Systèmes externes impliqués

ToulouseMap intervient sur le calcul d'itinéraires, ToulousePay sur le paiement. Les relations « **include** » et « **extend** » sont détaillées sur la slide suivante.

Relations entre cas d'utilisation

« include » déclenchement **obligatoire et systématique** — « extend » extension **conditionnelle**, hors chemin principal

Cas source	« include »	Justification
Saisir une demande de prise en charge (UC03)	S'authentifier (UC01)	Impossible de soumettre une demande sans être authentifié — inclusion non conditionnelle.
Valider le devis et payer (UC05)	Consulter le devis (UC04)	Le devis financier et écologique doit avoir été consulté avant tout paiement.
Accepter / refuser une mission (UC09)	Recevoir une proposition (UC08)	La décision n'existe qu'après réception d'une notification de proposition.
Scanner le QR Code de livraison (UC12)	Valider la collecte (UC11)	Le scan final suppose que la collecte du colis a été validée en amont.

Cas source	« extend »	Justification
Saisie manuelle du code (UC13)	Scanner le QR Code (UC12)	Proposée uniquement si le QR Code est endommagé ou illisible — extension conditionnelle.
Générer le ticket QR Code (UC06)	Valider le devis et payer (UC05)	N'a lieu qu'en cas de paiement réussi — prolonge le processus de validation.

UC détaillé n°1 — Demande de prise en charge et paiement

Acteurs : **Expéditeur** + ToulouseMap, ToulousePay — précondition : authentifié sur la plateforme web

- 1 Saisie des **adresses** (collecte, destination), du **colis** (poids, L×l×h) et de l'**urgence** (Standard 24h / Express 2h)
- 2 **ToulouseMap** évalue la distance réelle et la faisabilité du trajet
- 3 Calcul du devis : tarif au poids + **0,50 €/km** + majoration Express **+30 %**, et de l'**empreinte CO₂ économisée**
- 4 Affichage du **devis complet** — coût financier + CO₂ évité
- 5 Validation puis paiement sécurisé via **ToulousePay**
- 6 Commande **A_ATTRIBUER**, génération du **QR Code** et du **ticket d'expédition** imprimable

Paiement refusé

Processus suspendu, commande **VERROUILLEE_FRAUDE** — le client est invité à modifier son moyen de paiement.

Client refuse le devis

Retour au tableau de bord, aucune donnée persistante conservée.

Expiration de session

Déconnexion de sécurité en cas d'inactivité — les données saisies sont perdues.

UC détaillé n°2 — Accepter une mission et livrer

Acteur : **Livreur** + ToulouseMap — précondition : authentifié et déclaré **Disponible**

- 1 Le livreur active **Disponible** — sa position GPS est transmise **toutes les 30 s**
- 2 Le moteur d'attribution détecte un colis **à moins de 3 km** dans la file A_ATTRIBUER
- 3 **Notification** contextuelle (type, volume, distance) — **60 s** pour répondre
- 4 Acceptation → **EN_COURS_D_ATTRIBUTION** + **feuille de route optimisée** (pistes cyclables, ToulouseMap)
- 5 Collecte validée → **EN_COURS_DE_LIVRAISON**, livreur **En Tournée**
- 6 **Scan du QR Code** à destination → **LIVRE**, livreur de nouveau Disponible, récapitulatif affiché

Refus ou timeout (60 s)

Le colis retourne en file d'attente et est proposé au livreur disponible suivant.

QR Code endommagé

Procédure de secours : saisie manuelle du **code alphanumérique à 12 caractères** situé sous le QR Code.

Perte de réseau au scan

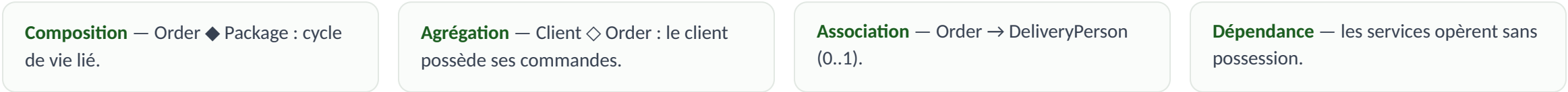
Événement stocké dans un **cache local chiffré**, synchronisation automatique au retour du réseau.



Entités, services et cycle de vie



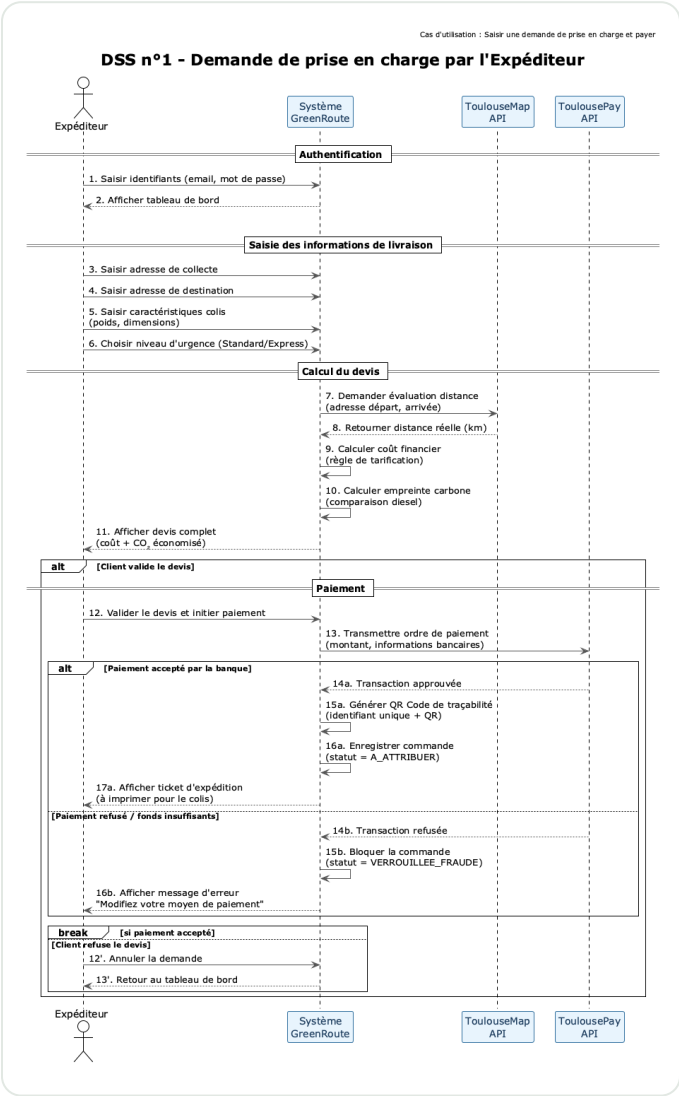
Relations du modèle



Cycle de vie d'une commande



DSS n°1 — Parcours expéditeur

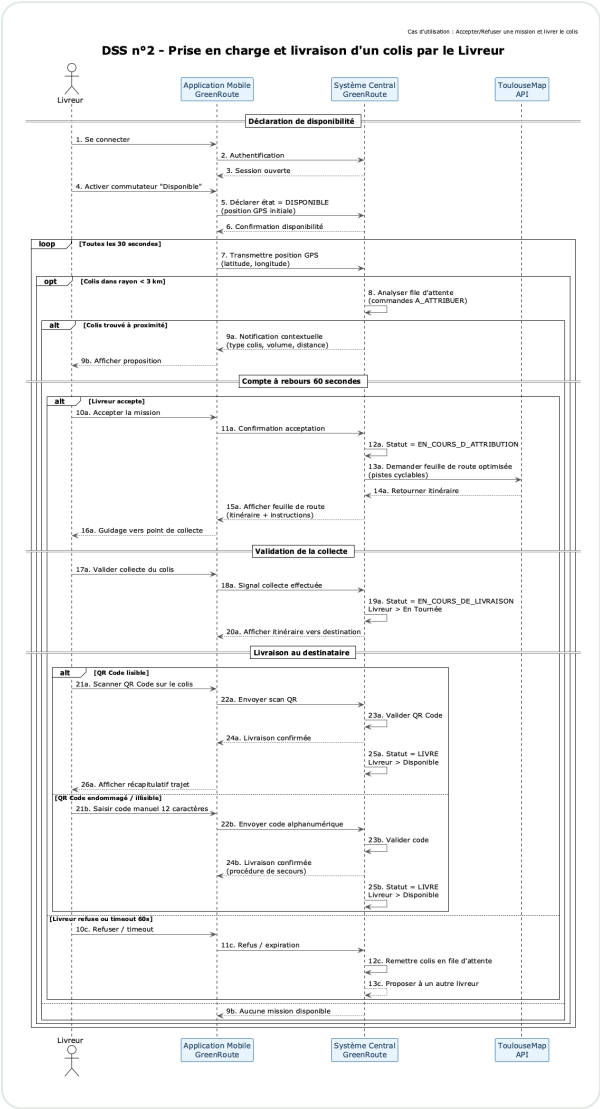


De l'authentification jusqu'au ticket d'expédition : saisie, devis, validation, paiement.

Fragment	Rôle dans le scénario
loop	L'expéditeur peut recommencer la saisie après une annulation de devis — plusieurs tentatives possibles.
alt	Devis : le client valide (poursuite vers le paiement) ou refuse (retour à la saisie).
alt	Paiement : ToulousePay accepte (A_ATTRIBUER + QR Code) ou rejette (VERROUILLEE_FRAUDE).
break	Sortie de la boucle de saisie dès qu'un paiement est accepté — fin du processus.

Chaque branche est modélisée avec ses postconditions — les deux issues du paiement sont explicites.

DSS n°2 — Parcours livreur



Connexion, disponibilité, GPS, notification, acceptation, collecte, livraison et scan.

Fragment	Rôle dans le scénario
loop	Transmission GPS toutes les 30 s tant que le livreur est disponible.
opt	Notification déclenchée uniquement si un colis est dans le rayon de 3 km.
alt	Proposition : accepter (nominal) ou refuser / timeout 60 s (remise en file).
alt	Scan final : QR lisible (scan) ou illisible (saisie manuelle du code à 12 caractères).

Les fragments combinés (loop, opt, alt, break) couvrent l'ensemble des exigences du cours.

Maquette n°1 — Interface web expéditeur

Scénario nominal

Demande de livraison & devis

Formulaire complet : adresses de collecte et destination, poids et dimensions du colis, sélecteur Standard / Express.

Devis calculé dynamiquement : **10,30 €** et **570 g de CO₂ économisés** — boutons « Valider et payer » / « Annuler ».

Scénario d'exception

Paieement refusé

Alerte rouge explicite : fonds insuffisants, plafond dépassé ou blocage sécurité — statut **VERROUILLEE_FRAUDE** affiché.

Actions correctives : « **Modifier mon moyen de paiement** » et « Contacter le support » — l'utilisateur garde le contrôle.

Maquette interactive : greenroute.cleeryy.cloud/mockups/expediteur.html — démonstration en direct pendant la soutenance

Maquette n°2 — Application mobile livreur

Scénario nominal

Notification de mission & scan QR

Carte de notification avec détails du colis (type, volume, poids, distance) et **compte à rebours de 60 s** — boutons Accepter / Refuser.

Section de scan QR Code avec animation de balayage pour valider la livraison.

Scénario d'exception

QR Code illisible

Encart d'erreur « QR Code illisible » et **procédure de secours** : saisie manuelle du code à 12 caractères (ex. : GR-7F3D-A2B9).

Message d'aide contextuel : « Le code se trouve sous le graphique du QR Code ».

Maquette interactive : greenroute.cleeryy.cloud/mockups/livreur.html — démonstration en direct pendant la soutenance

Critères ergonomiques de Bastien & Scapin

1. Guidage

Libellés explicites, code couleur vert/rouge, messages d'erreur avec actions correctives.

2. Charge de travail

Standard présélectionné, champs groupés par thème, le scan QR évite la saisie.

3. Contrôle explicite

Annulation possible avant paiement, choix binaire Accepter/Refuser, rien d'irréversible sans validation.

4. Adaptabilité

Scan ou saisie manuelle, cache local hors réseau, interface responsive.

5. Gestion des erreurs

Point fort : QR illisible → saisie manuelle, paiement refusé → changer de carte, perte réseau → cache chiffré.

6. Homogénéité

Vert #2E7D32 sur toutes les interfaces, navigation et terminologie uniformes.

7. Signifiante des codes

Vert = succès, rouge = erreur, orange = délai — compteur rouge sous 10 s.

8. Compatibilité

Vocabulaire métier logistique, représentations familières (QR, carte), parcours cognitif naturel.

Les 8 critères (1993) sont appliqués et argumentés en détail dans le DAF, avec exemples concrets par interface.

Implémentation Python du modèle

```
src/
├─ main.py          # démo bout en bout
├─ model/
│  ├─ enums.py      # statuts, urgence
│  ├─ client.py
│  └─ order.py      # classe centrale
├─ package.py
├─ delivery_person.py
└─ services/
   ├─ pricing_service.py
   ├─ routing_service.py
   ├─ payment_service.py
   └─ assignment_engine.py
```

Encapsulation stricte

Attributs privés (`_`), accès par `@property`, mutations via méthodes explicites (`set_disponible()`, `accepter_commande()`).

Choix techniques

Identifiants **UUID** (pas d'auto-incrément) · moteur d'attribution fondé sur la formule de **Haversine** (rayon 3 km) · relations composition / agrégation / association fidèles au diagramme.

Démonstration

`main.py` exécute le scénario nominal complet — du client à la livraison scannée — avec toutes les transitions de statut. **0 erreur de diagnostic.**

13 fichiers · 2 packages (`model`, `services`) · dépôt github.com/cleeryy/GreenRoute-UML

Répartition du travail & perspectives

Malik

Cadrage, besoins, cas d'utilisation et descriptions textuelles.

Swann

Architecture : diagramme de classes et diagrammes de séquence système.

Cléry

Maquettes IHM, critères ergonomiques et implémentation Python.

Couverture du barème

Attendu	Points
Analyse des besoins (contexte, UC, descriptions)	4 pts
Architecture (classes, DSS)	5 pts
IHM & code (maquettes, critères, Python)	5 pts
Qualité du livrable & soutenance	6 pts

Pistes pour une V2

- Optimisation de tournées multi-colis
- Scoring des livreurs et fiabilité
- Dashboard analytics pour la métropole
- Suivi temps réel côté destinataire

Merci de votre attention

Place à vos questions

Dépôt GitHub — github.com/cleeryy/GreenRoute-UML

DAF, diagrammes & maquettes — greenroute.cleeryy.cloud

Malik · Swann · Cléry — ESGI UML2 2025/2026



Code source



Présentation & maquettes